

# 6교시: 서비스 간 통신 확인

Week 3 Day 1 Lesson 6

Docker Compose로 전체 서비스 실행

단 한 번의 명령으로 개발 스택 전체를 빌드하고 실행합니다.

명령 실행 흐름

1 전체 서비스 빌드

build

\$ docker compose up --build -d

이미지가 없으면 빌드하고, 컨테이너를 데몬 모드로 실행

2 실행 중인 서비스 확인

ps

\$ docker compose ps

모든 서비스의 상태와 포트를 확인

3 API 상태 확인 (헬스 체크)

curl

\$ curl http://localhost:8080/api/status

프론트엔드를 통해 API /api/status 엔드포인트 호출

4 로그 확인

logs

\$ docker compose logs -f --tail=100

모든 서비스 로그를 실시간으로 확인

5 종료 및 정리 (필요 시)

cleanup

\$ docker compose down

\$ docker compose down -v

컨테이너 중지 및 네트워크/볼륨 정리 (-v는 볼륨까지 삭제)

서비스 상태 (docker compose ps 결과)

frontend

running

포트: 3000:80

역할: Web UI

api

healthy

포트: 8080:8080

역할: API Server

db

healthy

포트: 5432:5432

역할: PostgreSQL

worker

running

(내부 포트)

역할: 백그라운드 작업

docker compose ps (예시)

| NAME     | IMAGE              | COMMAND                  | SERVICE  | STATUS  | PORTS                  |
|----------|--------------------|--------------------------|----------|---------|------------------------|
| frontend | paperclip-frontend | "nginx -g 'daemon off;'" | frontend | running | 0.0.0.0:3000->80/tcp   |
| api      | paperclip-api      | "uvicorn main:app ...."  | api      | healthy | 0.0.0.0:8080->8080/tcp |
| db       | postgres:15        | "docker-entrypoint.s..." | db       | healthy | 0.0.0.0:5432->5432/tcp |
| worker   | paperclip-worker   | "python worker.py"       | worker   | running |                        |

curl /api/status (예시)

```
{
  "status": "ok",
  "service": "api",
  "version": "1.0.0",
  "timestamp": "2025-05-20T09:30:00Z"
}
```

docker compose logs -f (예시)

```
api      | INFO: Started server process [1]
api      | INFO: Application startup complete.
db        | 2025-05-20 09:30:00 LOG: database system is ready
worker   | [2025-05-20 09:30:05] Job queue connected
frontend | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty
```

빠른 명령 요약

1. build & up

2. ps (상태 확인)

3. curl (상태 체크)

4. logs (로그 확인)

5. cleanup (종료/정리)

docker compose up --build -d

docker compose ps

curl http://localhost:8080/api/status

docker compose logs -f

docker compose down

TIP

- d 옵션: 데몬(백그라운드) 모드로 실행
- build 옵션: 로컬 변경사항을 반영하여 이미지 재빌드
- ps: 현재 상태와 포트 매핑을 빠르게 확인
- logs -f: 실시간 로그 스트리밍으로 문제 빠르게 파악
- down -v: 볼륨까지 정리하여 깨끗한 상태로 재시작

## 수업 목표

- frontend -> api, api -> db, worker -> api 흐름을 각각 다른 증거로 확인한다.
- request id가 여러 service 로그를 이어주는 이유를 설명한다.
- host 기준 확인과 container 내부 기준 확인을 구분한다.

## 준비

```
cd week3/day1/labs/msa-demo
docker compose up --build -d
docker compose ps
```

## 흐름 1: frontend -> api

host에서 frontend를 통해 API를 호출한다.

```
curl -s -H 'x-request-id: w3d1-front-001' http://localhost:18083/api/status
```

확인할 것:

| JSON key           | 의미                       |
|--------------------|--------------------------|
| service            | 실제 응답한 service는 api      |
| request_id         | frontend가 전달한 request id |
| frontend_to_api    | frontend 경유 요청 성공 표시     |
| database_reachable | API가 DB까지 연결 가능한지        |

로그 확인:

```
docker compose logs --tail=80 api | grep w3d1-front-001 || true
```

grep에 `|| true`를 붙인 이유는 해당 request id가 없을 때도 실습 흐름을 계속하기 위해서다. 하지만 실제 장애 분석에서 무조건 붙이면 실패를 숨길 수 있다.

## 흐름 2: api -> db

API health endpoint를 직접 호출한다.

```
curl -i http://localhost:18084/health
```

정상 예:

```
HTTP/1.0 200 OK
```

본문에는 다음 값이 있어야 한다.

```
{
  "ready": true,
  "db_host": "db",
  "db_port": 5432,
  "error": null
}
```

여기서 db\_host가 localhost가 아니라 db라는 점을 강조한다. API container 입장에서 localhost는 API 자기 자신이다.

## 흐름 3: worker -> api

worker는 host port가 없으므로 curl로 직접 호출하지 않는다. log로 확인한다.

```
docker compose logs --tail=80 worker
```

정상 worker log 형태:

```
{
  "service": "worker",
  "request_id": "worker-...",
  "api_url": "http://api:8080/api/status",
  "status": 200
}
```

worker가 http://localhost:8080을 쓰면 실패한다. worker container 내부의 localhost는 worker 자신이기 때문이다.

## host 기준과 container 기준

| 관점                       | 주소 예시                      | 사용하는 상황                   |
|--------------------------|----------------------------|---------------------------|
| host -> frontend         | http://localhost:18083     | browser/curl로 사용자 경로 확인   |
| host -> api debug        | http://localhost:18084     | 강의에서 API health 직접 확인     |
| host -> catalog-api      | http://localhost:18120     | 상품 API 직접 확인              |
| host -> order-api        | http://localhost:18121     | 주문 API 직접 확인              |
| frontend -> api          | http://api:8080            | nginx reverse proxy 내부 통신 |
| api -> db                | db:5432                    | API container 내부 DB 연결    |
| worker -> api            | http://api:8080/api/status | background service 통신     |
| order-api -> redis       | redis:6379                 | 주문 이벤트 queue 입력           |
| order-worker -> redis/db | redis:6379, db:5432        | 주문 이벤트 처리와 상태 변경          |

## 확장 흐름: catalog/order 서비스

기존 frontend, api, worker, db 흐름은 그대로 둔다. 여기에 더 실제 서비스에 가까운 API를 같은 Compose stack에 추가해서 service가 늘어났을 때 통신 증거가 어떻게 달라지는지 본다.

추가 service:

| Service      | 역할                | host 확인         | 내부 dependency           |
|--------------|-------------------|-----------------|-------------------------|
| catalog-api  | 상품 목록 API         | localhost:18120 | db                      |
| order-api    | 주문 생성/조회 API      | localhost:18121 | db, redis               |
| redis        | 주문 이벤트 queue      | host 공개 없음      | order-api, order-worker |
| order-worker | queue에서 주문 이벤트 처리 | host 공개 없음      | redis, db               |

확장 요청 흐름:

```
host -> catalog-api -> db
host -> order-api -> db
order-api -> redis queue
order-worker -> redis queue -> db
```

상품 조회:

```
curl -s -H 'x-request-id: w3d1-catalog-001' \
  http://localhost:18120/api/catalog/products
docker compose logs --tail=60 catalog-api | grep w3d1-catalog-001 || true
```

주문 생성:

```
curl -s -X POST -H 'x-request-id: w3d1-order-001' \
  http://localhost:18121/api/orders
docker compose exec redis redis-cli LLEN order-events
docker compose exec redis redis-cli LRANGE order-events 0 -1
docker compose logs --tail=80 order-api | grep w3d1-order-001 || true
docker compose logs --tail=80 order-worker | grep w3d1-order-001 || true
```

order-worker는 교육용으로 ORDER\_WORKER\_POLL\_SECONDS=2초마다 queue를 확인하고, message를 발견하면 ORDER\_WORKER\_MESSAGE\_VISIBILITY\_SECONDS=10초 동안 일부러 queue에 남겨둔 뒤 소비한다. 그래서 주문을 넣은 직후에는 Redis list에 message가 잠깐 남아 있어야 한다.

| 명령                       | 의미                           |
|--------------------------|------------------------------|
| LLEN order-events        | 현재 queue에 남아 있는 message 개수   |
| LRANGE order-events 0 -1 | queue에 남은 message payload 확인 |
| logs order-worker        | worker가 message를 꺼내 처리했는지 확인 |

LRANGE 결과 예시:

```
{"order_id":12,"request_id":"w3d1-order-001"}
```

약 10초가 지나면 worker가 message를 꺼내므로 LLEN은 다시 0이 된다. 이 변화가 queue에 들어감 -> worker가 소비함 -> DB 상태 변경 흐름의 핵심 evidence다.

주문 상태 확인:

```
curl -s http://localhost:18121/api/orders
```

queue가 처리되었는지 다시 확인:

```
docker compose exec redis redis-cli LLEN order-events
```

DB audit log 확인:

```
docker compose exec db psql -U paperclip -d paperclip \
  -c "select service_name, request_id, event, created_at from audit_logs order
  by id desc limit 5;"
```

여기서 중요한 점은 order-api가 사용자 요청을 받지만, 실제 주문 처리는 order-worker 로그와 DB 상태 변경까지 봐야 완료된다는 것이다. W3D2의 장애 전파 수업에서는 redis 또는 order-worker를 멈춰서 “API 응답과 실제 처리 완료는 다를 수 있다”를 확인할 수 있다.

## 통신 실패를 일부러 만들기 전 질문

장애를 넣기 전에 먼저 정상 경로를 말할 수 있어야 한다.

사용자 요청은 localhost:18083으로 들어온다.  
 frontend container는 api:8080으로 요청을 넘긴다.  
 api는 db:5432로 PostgreSQL 연결을 확인한다.  
 worker는 api:8080/api/status를 주기적으로 호출한다.  
 order-api는 redis:6379에 주문 이벤트를 넣는다.  
 order-worker는 redis queue에서 이벤트를 꺼내 db:5432에 처리 결과를 남긴다.

이 네 문장을 말하지 못하면 장애 분석은 운에 맡기는 일이 된다.

## Evidence Note

```
# W3D1S6 Communication Evidence
- frontend -> api curl:
- api -> db health:
- worker -> api log:
- catalog-api -> db curl/log:
- order-api -> redis -> order-worker -> db evidence:
- host address vs service name difference:
```

## 핵심 포인트

MSA의 통신 확인은 curl 한 번 성공이 아니다. 사용자 경로, service 내부 경로, background 경로를 나눠서 각각의 증거를 잡는 것이다.